Team Contribution Journal

Compiled member entries

ITAI 2376 · Creating Images with Diffusion Models

Compiled from supplied member entries. Formatting and completeness checks do not establish authorship, factual correctness, or author approval.

Team Contribution Journal

Michael

Responsibility and work completed

I constructed the project code and organized it into notebook sections for the team to complete. After my teammates returned their completed notebooks, I brought their work together, added the CLIP integration, and assembled the final deliverables.

Implementation and integration

My contribution covered the initial code construction and the final integration of the team's returned work. I connected the completed sections into the consolidated notebook and organized the final assessment around the training records, generated images, and evaluation evidence.

I performed the CLIP integration as part of the final assembly. The CLIP experiment and the MNIST classifier evaluation are separate checks, so their results must be reported using their own definitions rather than combined into one accuracy or image-quality score.

Final documentation and deliverables

I assembled the notebook, assessment report, contribution journal, and reflection journal for submission. This included organizing the team's existing written entries and checking that the generated documents contained actual content rather than unfinished templates.

Collaboration

I provided the code structure, received the completed notebook sections from my teammates, and compiled their returned work into the final deliverable. Their entries document the work they completed, investigated, explained, and returned; my entry documents the code construction and final integration.

Supporting evidence

The supporting evidence consists of the project code, the returned notebook sections, the consolidated notebook, the Part 4 CLIP integration, and the final submission documents. Numerical claims about CLIP and model performance should reference the selected executed results, not be inferred from the presence of code alone.

Cherluk

Source: `part1_handoff.json`. Existing statements are preserved as source excerpts; author approval is not inferred.

Work log: stage1

What i did

> I first converted the MNIST images into tensors and normalized the pixel
> values to [-1, 1]. I then split the dataset into 80% training and 20%
> validation using seed 42. After that, I created the training and validation
> DataLoaders, shuffled the training data, and kept the validation data in a
> fixed order. Finally, I checked the image shapes, labels, pixel range, split,
> and class balance before moving forward.

Decisions and why

> I used an 80/20 train-validation split because it gives the model enough data
> for training while still keeping 12,000 images for validation. I used seed 42
> so the split can be reproduced by the rest of the team. I also shuffled only
> the training loader because the model should receive mixed training batches,
> while validation does not need to be shuffled.

Bugs or surprises

> I did not run into a major bug in this stage. One thing I needed to understand
> was why normalization changed the background pixels close to -1 instead of 0.
> After looking at the normalization formula, I understood that values are being
> shifted from [0, 1] to [-1, 1], so this result was expected.

Evidence it works

> The verification output showed that the image shape, pixel range, dataset
> split, labels, shuffling, and reproducible split all passed their checks.
> I also used the displayed MNIST images and class-balance graph as visual
> evidence that the data was loaded and split correctly. These checks gave me
> confidence that the data pipeline is ready for the U-Net and the next part
> of the project.

Work log: stage2

What i did

> I built the main parts of the U-Net by completing the ConvBlock, UpBlock, and
> the conditioning and decoder section of UNet.forward. I connected the encoder
> and decoder using skip connections and added both time and class conditioning
> to the middle features. After building the model, I ran the provided checks
> and shape trace to make sure information moved through the network correctly.

Decisions and why

> I followed the U-Net design in the notebook so the image could be downsampled
> to learn deeper features and then upsampled back to its original size. I
> concatenated the skip connections along the channel dimension because this
> keeps the encoder features while combining them with the decoder features.
> I also reshaped the time and class conditioning to (batch, 128, 1, 1) so they
> could be added across the spatial dimensions of the middle feature map.

Bugs or surprises

> One part that required more attention was understanding how the number of
> channels changed after a skip connection was concatenated with the upsampled
> tensor. I initially had to trace the shapes carefully to see why up1 receives
> 192 channels instead of 64 or 128. Following the shape trace made it clear
> that the 64 upsampled channels and 128 skip channels are combined together.

Evidence it works

> The verification checks showed that the output has the same shape as the
> input and that changing either the timestep or class changes the output. The
> skip ablation checks showed that both skip connections influence the result,
> and the gradient check showed that every parameter receives a gradient. The
> shape trace also showed the expected encoder and decoder dimensions, ending
> with an output shape of (4, 1, 28, 28). Together, these checks gave me evidence
> that the U-Net architecture is connected and working as intended before training.

Olu

Source: `part2_handoff.json`. Existing statements are preserved as source excerpts; author approval is not inferred.

Work log: stage1

What i did

> I built the 100-step schedule, tested the forward equation, compared the standard schedule with the rescaled schedule, added the forward-noise function, and checked the halfway timestep.

Decisions and why

> I kept the 100-step setup required by the notebook and rescaled the beta range so the final state is almost pure static.

Bugs or surprises

> The standard 0.0001-to-0.02 values do not reach pure static when compressed into only 100 steps. I also found that the first forward-process verification was run before predict_x0 had been defined, which caused a NameError even though the formula itself was correct.

Evidence it works

> The schedule checks passed, the toy forward calculation matched 0.6008 for its intended row, t=0 stayed close to the original, the final step had mean near 0 and standard deviation near 1, and the rescaled schedule made alpha_bar about 0.000020.

Work log: stage2

What i did

> I implemented the reverse-step mean and the conditional fresh-noise term, then tested shape, exact recovery at t=0, determinism at t=0, randomness for t>0, and the expected noise scale.

Decisions and why

> I used the formula exactly as given and applied the t>0 mask so the final step is deterministic.

Bugs or surprises

> The reverse step was easier to verify than the forward helper because the t=0 sanity check gives a very strong signal: a correct implementation nearly reconstructs the original exactly.

Evidence it works

> All five reverse-step checks passed, including a largest t=0 reconstruction error of only 1.55e-06.

Work log: stage3

What i did

> I implemented the train step, verified the expected untrained loss, checked gradient flow, and ran a repeated-batch optimization smoke test.

Decisions and why

> I used MSE between predicted and true noise because that is the training target defined by this notebook's diffusion formulation.

Bugs or surprises

> The untrained loss being close to 1 was expected rather than a sign of failure. The smoke test then dropped from 0.512 to 0.053, which gave strong evidence that the training path was connected.

Evidence it works

> All Stage 3 checks passed. The repeated-batch loss fell from 0.512 to 0.053, weights changed, no NaNs appeared, and every model parameter received a gradient.

Work log: stage5

What i did

> I trained the verified U-Net for 20 epochs, saved the best checkpoint, inspected training and validation curves, compared snapshots, measured loss at every timestep, tested one-shot x0 recovery, and ran timestep/class ablations.

Decisions and why

> I kept the 100-step rescaled schedule and learning rate of 1e-3 because the run already converged strongly and all Stage 4 verification checks passed.

Bugs or surprises

> The biggest surprise was that the highest per-timestep loss occurred at t=0 rather than at high noise. Another real notebook-order issue caused the predict_x0 verification to fail with NameError before the function had been executed. The export step also initially missed the forward function because its export tag was absent.

Evidence it works

> The 20-epoch run reduced training loss from 0.0687 to 0.0234 and validation loss to 0.0230. All 10 Stage 4 checks passed, the checkpoint reloaded correctly, generated samples had the expected shape and finite range, and the wrong-timestep ablation increased loss from 0.0233 to 0.0542.

Taki

Source: `/home/ubuntu-user/Documents/MidtermDoc(DeepLearning)/FINAL_SUBMISSION/supporting_files/part3_results.json`.

Recorded contribution

> I completed the generation and evaluation section after receiving Olu's trained diffusion checkpoint. I generated all ten MNIST classes, examined repeated samples, visualized the reverse denoising process, tested class conditioning with controlled starting noise, and evaluated generated images with an independent MNIST classifier.